Java Assignment - 2

- **Name:** Saloman Raju Sarikokkula
- **Date of Submission:** 21-03-2026

Date: 16-03-2026

- **Topics:** Data types & Variables

Data Types

- Data types are used to define the **type** and **size** of the value where a variable is going to store in it.
- Categorize into **Two** types:
    1. **Primitive:** byte, short, int, long, float, double, char, boolean.
    2. **Non-Primitive:** String, Array, Class, Interface, Collection.

1. Primitive Data Types

These are in-built or predefined data types.

Syntax:

```
<data_type> <variable> = <value>
```

Example:

```
int number = 20;
```

Program:

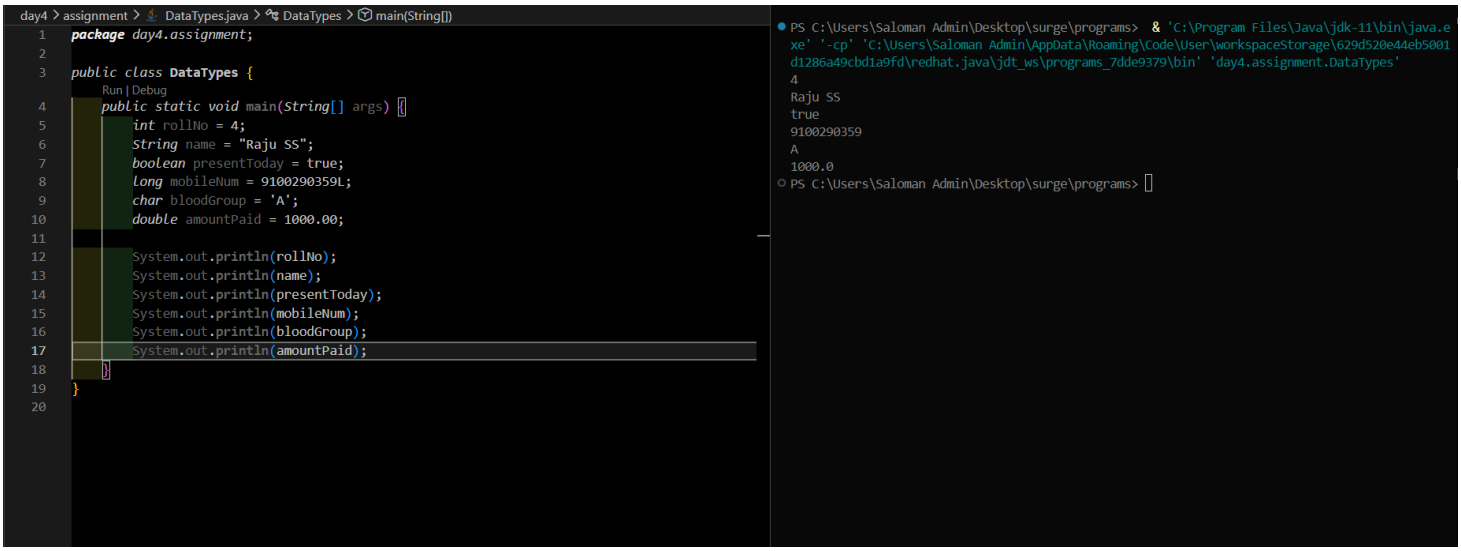
```
package day4.assignment;

public class DataTypes {
    public static void main(String[] args) {
        int rollNo = 4;
```

```
        boolean presentToday = true;
        long mobileNum = 9320392369L;
        char bloodGroup = 'A';
        double amountPaid = 1000.00;

        System.out.println(rollNo);
        System.out.println(presentToday);
        System.out.println(mobileNum);
        System.out.println(bloodGroup);
        System.out.println(amountPaid);
    }
}
```

Execution in VSCode:



2. Non-Primitive Data Types

- These are the data types which store references rather values.
- That's why they are also called as **Reference Data Types**
- **Types:** String, Array, Class, Interface, Collection, .etc.

Syntax:

```
<data_type> <ref_variable> = new <data_type>([vals]);
```

Example:

```
String name = new String("phantom");
```

Variables

- Variables are the containers which are used to store the values during the program execution.
- These are **Three** types:
    1. Local Variables
    2. Instance Variables
    3. Static Variables

1. Local Variables

- The variables which are defined inside the methods are called Local variables.

- They don't have default values.

Example:

```
public class Sample {

    public void sample () {          // method definition
        String name = "Dall-E";      // local variable
    }

}
```

2. Instance Variables

- The non-static variables which are defined inside a class but outside the methods are called Instance Variables or Non-static Field.
- Each object gets their separate instance of these instance variables.

Example:

```
public class Employee
{
  int employeeId;  //Instance Variable OR Non-Static field
}
```

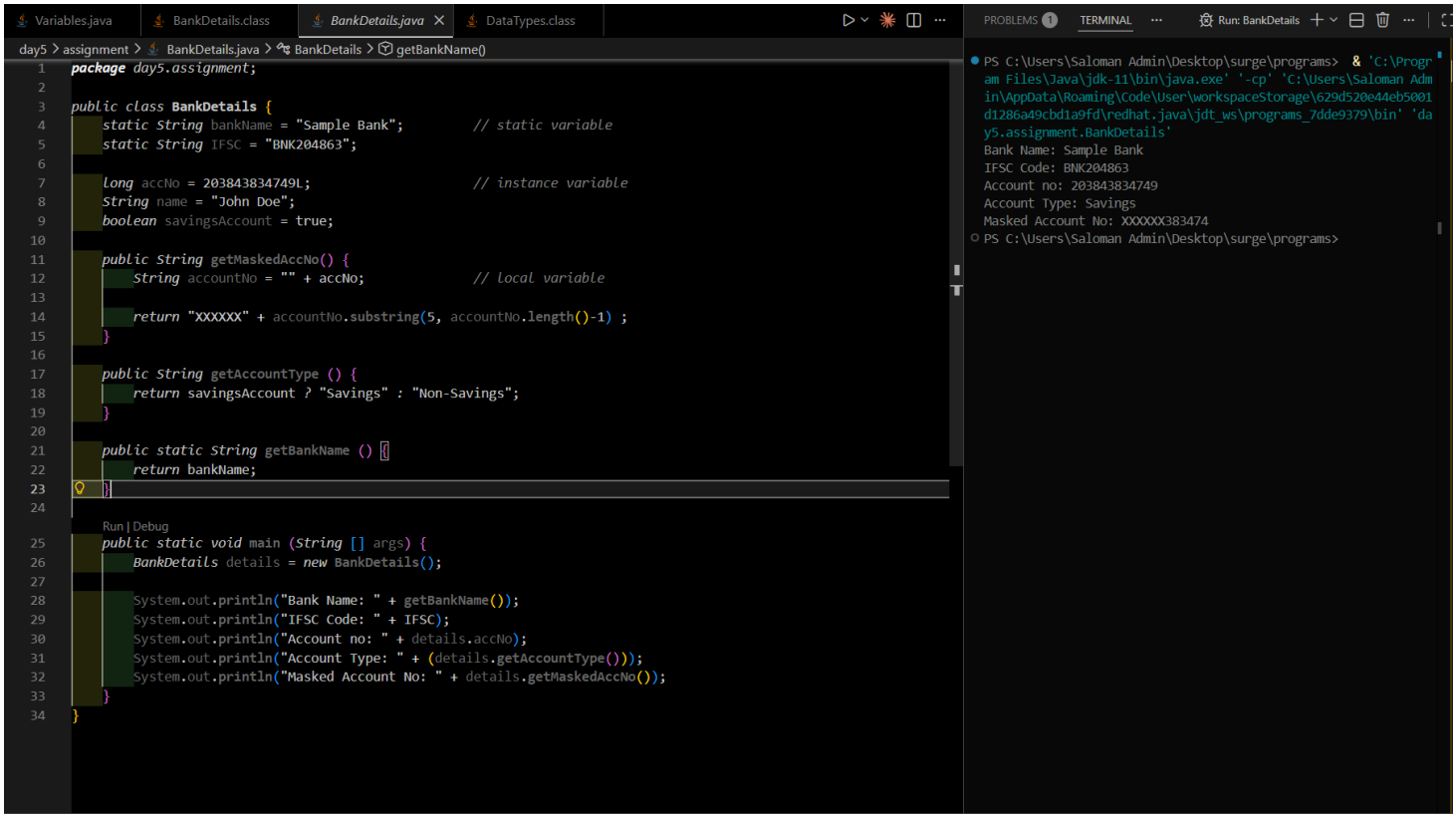
3. Static Variables

- The static variables belongs to class and only one copy shared across all the objects.
- The word `static` is used to define them.

Example:

```
public class Employee
{
    static String companyName;   // Static variable
}
```

Execution in VSCode:



Date: 17-03-2026

- **Topics:** Methods & Constructor

Methods

- A block of code which is used to perform various operations or printing data.
- It promotes the `code reusability`,as it only defines once, but can be called multiple times.
- These are **two** types:
    1. Pre-defined or In-built methods
    2. User-defined or Custom methods

1. Predefined Method

- A method which is developed by the language developer itself.
- No need to define these methods explicitly, but can be called multiple times.

Example: `String.length()`, `String.substring()`, etc.

2. User- defined Method

- A method defined by user based on their own requirements or specifications.
- It has two parts:
  a) Method Definition
  b) Method Calling

Syntax:

```
class Sample
{
    <access_modifier> [static] <return_type> <someMethodName>([params]) {
                                                        // method definition
        ..operations
        ..
        .
    }

    public void anyOtherMethod () {
        <someMethodName>(params);                 // method calling
    }
}
```

Example:

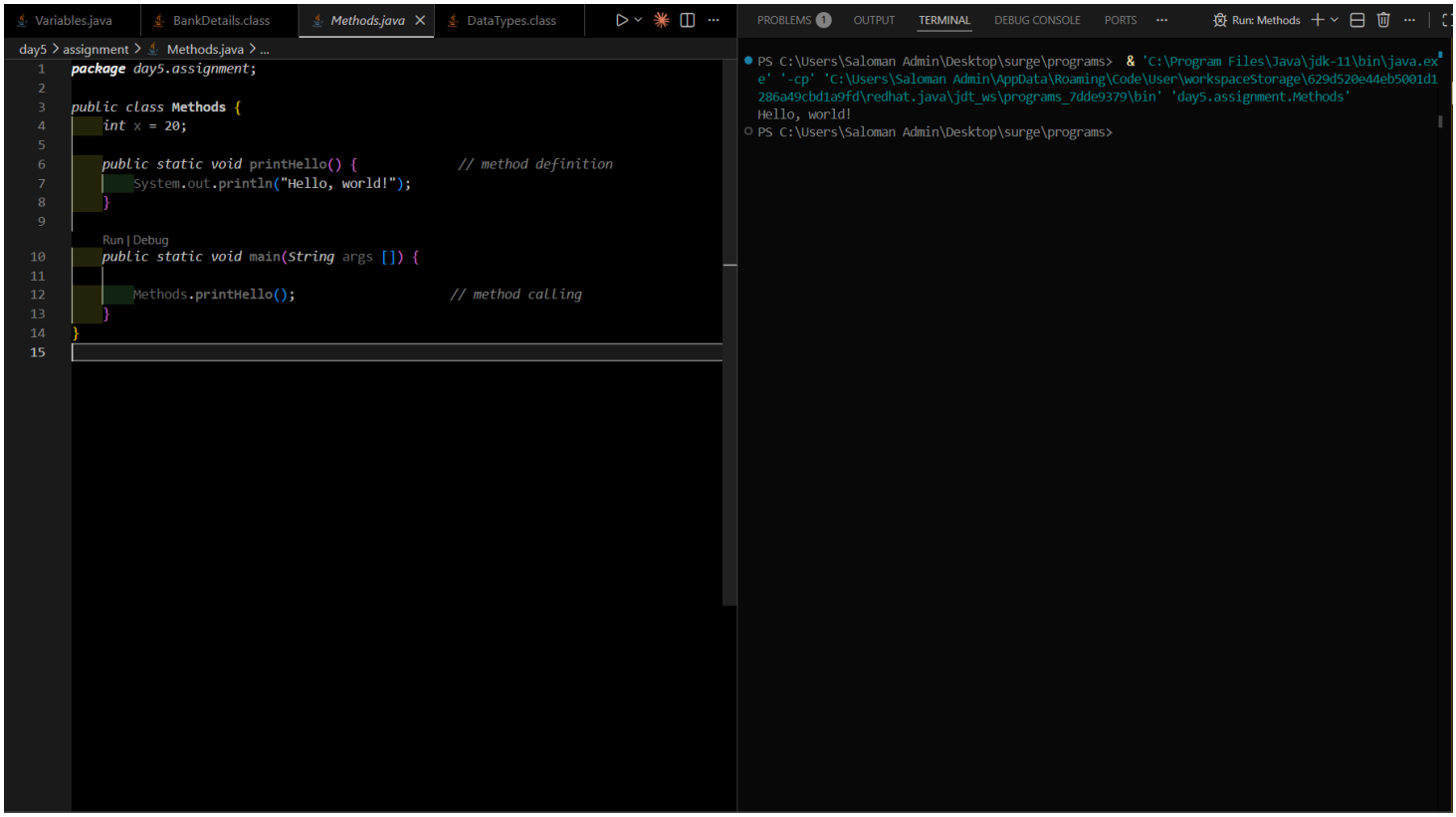
```
public class Methods {
    int x = 20;

    public static void printHello() {              // method definition
        System.out.println("Hello, world!");
    }

    public static void main(String args []) {

        Methods.printHello();                     // method calling
    }
}
```

Execution in VSCode:



Constructor

- It is used to construct object, that's why it's called as Constructor.
- The main purpose is to initialize the object properties with user-defined values.
- It is a method with name same as the class name, but without any return type.
- There are **three** types of constructors:
    1. **Default No Argument Constructor:** Added by compiler.
    2. **Non-Parameterized Constructor:** Added by user, but have no arguments.
    3. **Parameterized Constructor**: Added by user, but have parameters.

Syntax:

```
class ClassName
{
    [accessModiefier] ClassName() {        // constructor
        ..
        ..
    }
}
```

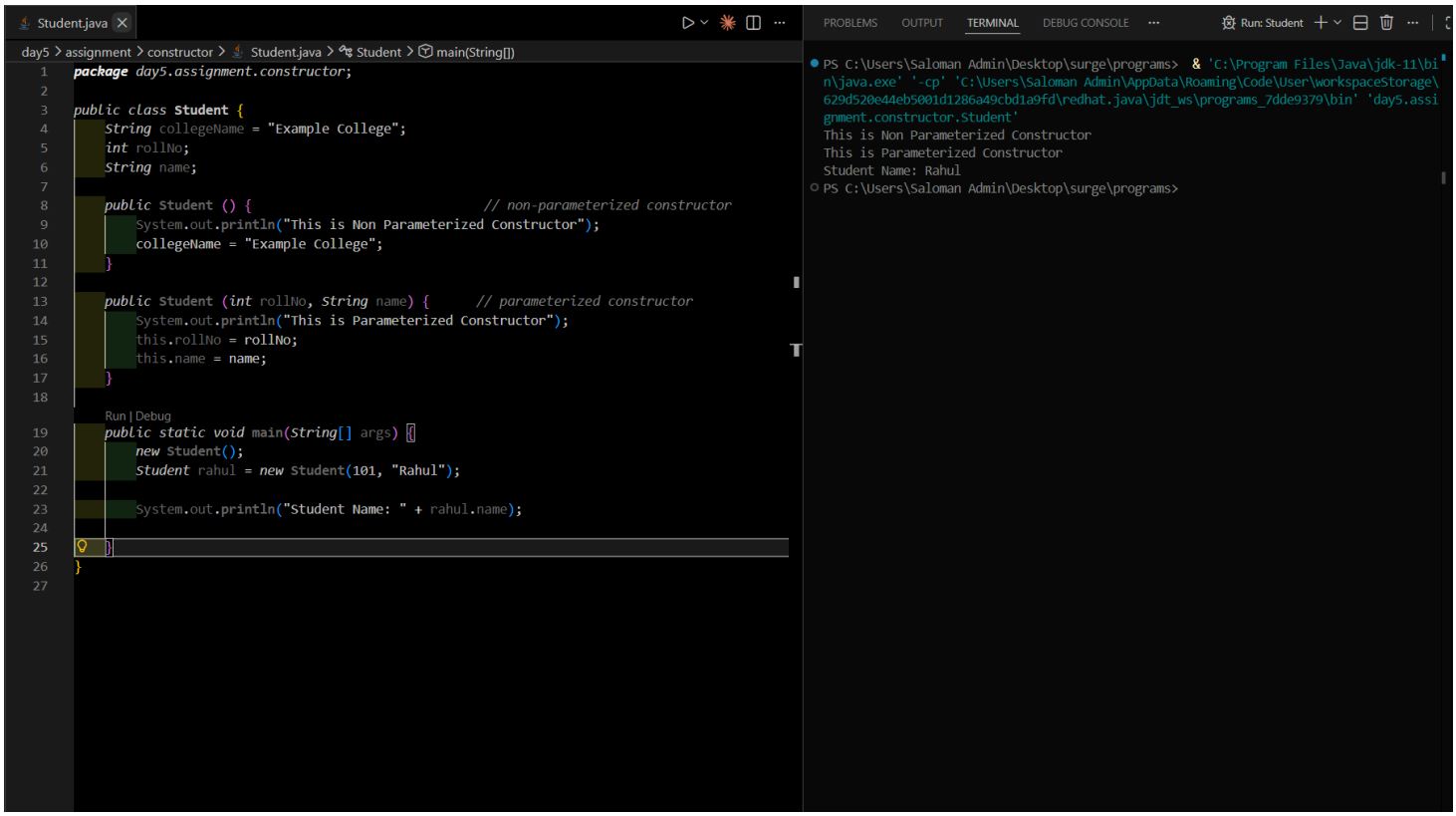
Example:

```
class Student
{
  String name;

  public Student(String name)    // constructor
  {
      this.name = name;
      return;
  }

  public static void main(String [] args)
  {
     Student s1 = new Student();
  }
}
```

Execution in VSCode:



Date: 18-03-2026

- **Topics:** Scanner Class & Operators

Scanner Class

- It is pre-defined class available in `java.util` package introduced from jdk 1.5v.
- It is used to read the data or input from the user.
- Methods of Scanner class:
  - public String next(): To read a single word.
  - public String nextLine(): To read multiple words or complete line.
  - public byte nextByte(): To read byte value.
  - public short nextShort(): To read short value.
  - public int nextInt(): To read int value.
  - public long nextLong(): To read long value.
  - public float nextFloat(): To read float value.
  - public double nextDouble(): To read double value.
  - public boolean nextBoolean(): To read boolean value.
  - public char next().charAt(0): To read a single char value.

Syntax:

```
import java.util.Scanner;

class Sample {
    public static void main (String args [])
    {
        //object creation for Scanner class
        Scanner sc = new ScannerSystem.in);

        int x = sc.nextInt();     // reads int value
    }
}
```

Example:

```
import java.util.Scanner;

public class ReadInput {
```

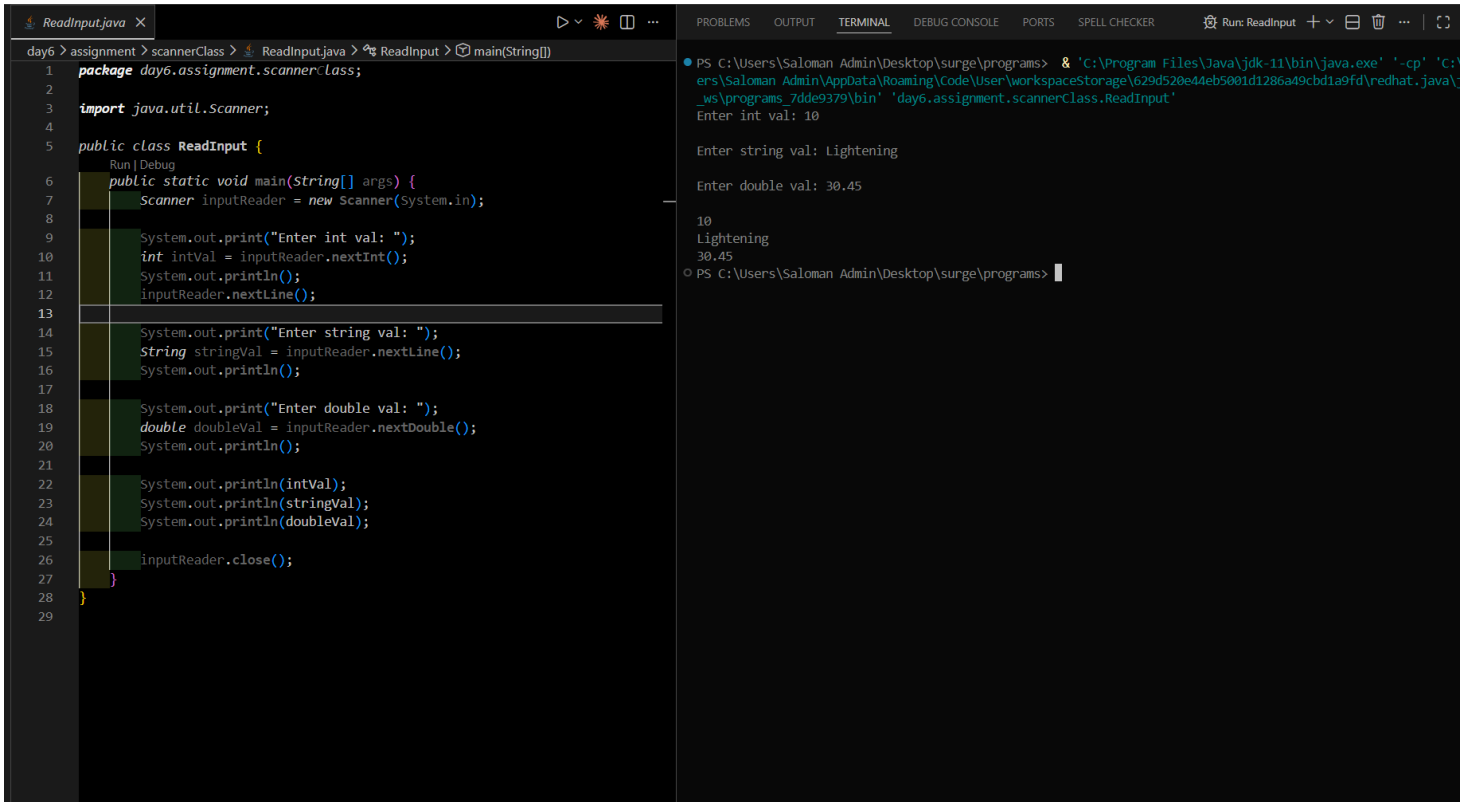
```
    public static void main(String[] args) {
        Scanner inputReader = new Scanner(System.in);

        System.out.print("Enter int val: ");
        int intVal = inputReader.nextInt();
        System.out.println();

        System.out.println(intVal);

        inputReader.close();
    }
}
```

Execution in VSCode:



Operators

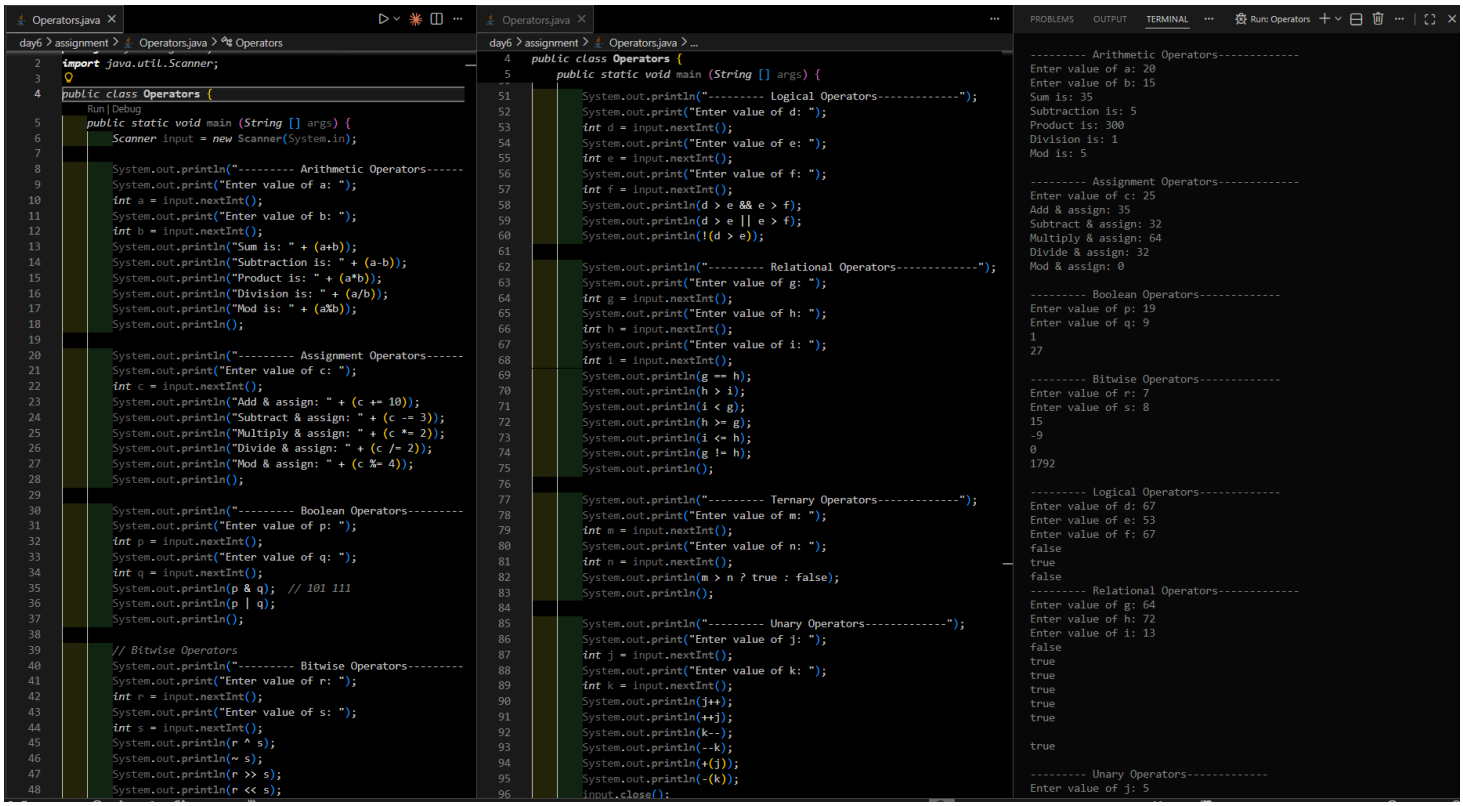
- An operator is a symbol used to perform some operations on operands.
- Types of Operators:
    1. Arithmetic
    2. Assignment
    3. Logical
    4. Relational
    5. Unary

6.Boolean

7.Ternary

S.No	Operator	Symbols	Examples
1.	Arithmetic	+, -, *, /, %	a + b, a-b, a*b
2.	Assignment	=, +=, *=,/=, %=	a = 10, a+= 20
3.	Logical	&&, ||, !	(a>b) && (a<b)
4.	Relational	>, <, >=, <=, !=	a > b, c <= d
5.	Unary	++, —	a++, —b
6.	Boolean	&, |	a & b, a | c
7.	Unary	expr ? true : false	n>2 ? true: false

Execution in VSCode:



Date: 20-03-2026

- **Topics:** Conditional Statements & Comments

Conditional Statements

- Conditional Statements allow the user to control the flow of the program by executing certain

block of code only if specific condition is true.

- Types:
    1. Simple -if
    2. if-else
    3. if-else if-else
    4. switch statement

1. simple - if

Executes a block of code only if certain condition is true.

Syntax:

```
if (condition or expression) {
    statement1
    statement2
    ...
}
```

Example:

```
int age = 18;

if (age >= 18) {
    System.out.println("You are eligible to vote.");
}
```

2. if - else

Executes one block of code only if certain condition is true, otherwise executes another block of code.

Syntax:

```
if (condition or expression) {
    statement1
    statement2
    ...
}
else {
    statement1
    statement2
    ...
```

```
}
```

Example:

```
int age = 18;

if (age >= 18) {
    System.out.println("You are eligible to vote.");
}
else {
    System.out.println("You are NOT eligible to vote.");
}
```

3. if-else if-else

Checks multiple conditions in sequence and executes the respective block whose condition is true.

Syntax:

```
if (condition or expression) {
    statements
    ...
}
else if (condition or expression) {
    statements
    ...
}
else {
    statements
    ...
}
```

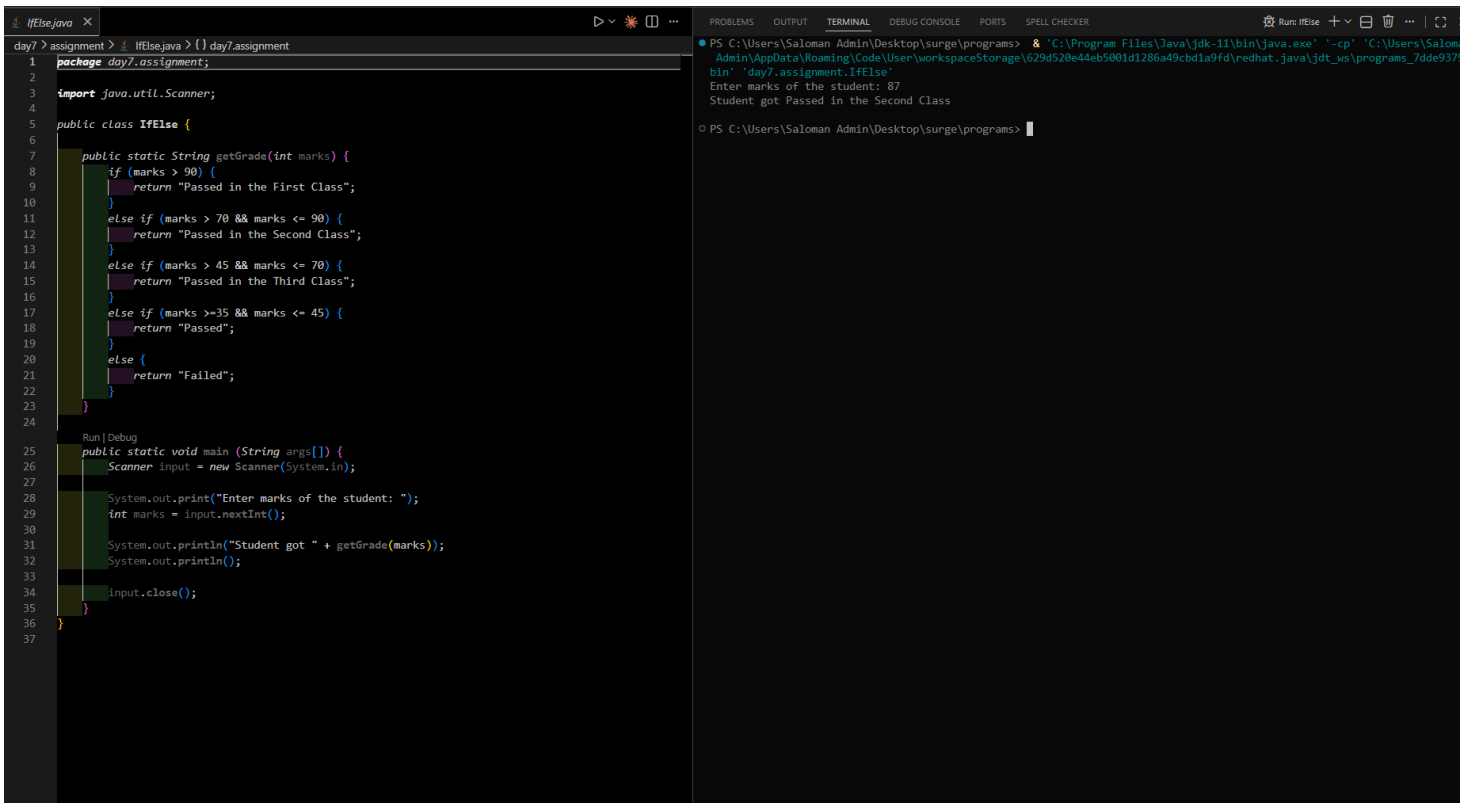
Example:

```
int marks = 85;

if (marks >= 90) {
    System.out.println("Grade: A");
} else if (marks >= 80) {
    System.out.println("Grade: B");
} else if (marks >= 70) {
    System.out.println("Grade: C");
```

```
} else {
    System.out.println("Grade: D");
}
```

Execution in VSCode:



4. switch statement

Selects one of the blocks of code based on the value and executes it.

Syntax:

```
switch (valuex) {
    case val1: { stmts
                 break;
                 }
    case val2: { stmts
                 break;
                 }
```

Example:

```
int day = 3;

switch (day) {
    case 1:
        System.out.println("Monday");
```

```
            break;
        case 2:
            System.out.println("Tuesday");
            break;
        case 3:
            System.out.println("Wednesday");
            break;
        default:
            System.out.println("Invalid day");
    }
```

Execution in VSCode:

Calculator.java

day7 > assignment > Calculator.java > ...

```java
package day7.assignment;

import java.util.Scanner;

public class Calculator {
    public static void calculator(int option, int n1, int n2) {
        switch (option) {
            case 1:
                System.out.println("Result is: " + (n1 + n2));
                break;
            case 2:
                System.out.println("Result is: " + (n1 - n2));
                break;
            case 3:
                System.out.println("Result is: " + (n1 * n2));
                break;
            case 4:
                System.out.println("Result is: " + ((double) n1 / (double) n2));
                break;
            case 5:
                System.out.println("Result is: " + (n1 % n2));
                break;
            default:
                System.err.println("Select a valid option");
        }
    }

    public static void exitCalculator() {
        System.out.println("Exited Successfully");
        System.exit(0);
    }

    Run | Debug
    public static void main(String args[]) {
        Scanner input = new Scanner(System.in);

        int option = 0;

        do {
            System.out.print("Enter the value of 1st num: ");
            int n1 = input.nextInt();

            System.out.print("Enter the value of 2nd num: ");
            int n2 = input.nextInt();

            System.out.println();
            System.out.println("1. Addition\n2. Subtraction\n3. Multiplication\n4. Division\n5. Modulus\n6. EXIT");
            System.out.print("Select an option bw 1 to 6: ");
            option = input.nextInt();
            if (option < 1 && option > 6) {
                System.err.println("Select an option bw 1 to 6 only.");
            } else {
                if (option == 6) {
                    exitCalculator();
                }
                calculator(option, n1, n2);
            }
            System.out.println();
            System.out.println("1. Do more operations\n2. EXIT");
            System.out.print("Select either 1 or 2: ");
            int secOption = input.nextInt();

            if(secOption == 2) {
                exitCalculator();
            }
        } while (option > 0 && option < 6);

        input.close();
    }
}
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS

```
PS C:\Users\Saloman Admin\Desktop\surge\programs> &
dde9379\bin' 'day7.assignment.Calculator'
Enter the value of 1st num: 20
Enter the value of 2nd num: 40

1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Modulus
6. EXIT
Select an option bw 1 to 6: 1
Result is: 60

1. Do more operations
2. EXIT
Select either 1 or 2: 1
Enter the value of 1st num: 89
Enter the value of 2nd num: 43

1. Addition
2. Subtraction
3. Multiplication
4. Division
5. Modulus
6. EXIT
Select an option bw 1 to 6: 4
Result is: 2.0697674418604652

1. Do more operations
2. EXIT
Select either 1 or 2: 2
Exited Successfully
PS C:\Users\Saloman Admin\Desktop\surge\programs>
```

Comments

- Comments are statements whose execution is usually ignored by the compiler.
- These are helpful for code explanation and documentation.
- These are **three** types:
    1. Single line
    2. Muti-line
    3. Documentation

1. Single Line Comment

Used for short or single line explanations

Syntax and Example:

2. Multi-line Comments

Used for longer explanations that span multiple lines.

Syntax and Example:

```
/*
This is a multi-line comment.
It can span several lines.
*/

int calculateSum(int a, int b)
{
    return a + b;
}
```

3. Documents Comments

Used to generate API documentation. These comments are placed before classes, methods, or fields.

Syntax and Example:

```
/**
 * This method calculates the sum of two integers.
 * @param a the first integer
 * @param b the second integer
 * @return the sum of a and b
```

```
 */
public int add(int a, int b) {
    return a + b;
}
```

THE END